Principal of Conceptual Modeling

2026/11/07 - Version 3.5.0

> [!IMPORTANT] このドキュメントは、note.com のメンバーシップである、“Ordo Forma Est”のTOP記事、“概念モデリングの形式の普遍性を問う ～ 問答、議論のとっかかりとして”の”概念モデリングの体系”の部分を切り出し、更に、その後の考察も含め精緻化してきたものです。

> [!NOTE] note.com、github.com 共に、Texの変換が不十分です。それを補うため、PDF版を用意していますので、そちらのご参照を推奨します。

概念モデリングの体系

2026年8月現在の時点で到達した概念モデリングの体系を、ウィトゲンシュタインの論理哲学論考のスタイルを参考に列記していくことにします。ある項目を指し示したい場合は、インデックスのレベルに従って、”2-9-2”の様に明記していただくとありがたいです。

1. モデル化対象

1. この世界は、マルクスガブリエルの新実在論を論拠とした、複数（無限）の意味の場（Field of Sense）が並立しているものとする
2. それぞれの意味の場において、”モノ（Object）”が意味を伴って立ち現れている
3. 概念モデリングは、モノの意味の構造をモデル化する
4. 現実の世界の観察から、異なる意味の場は意味的には無関係であるが、一定の対応関係をもって現象している
5. よって、概念モデリングは、意味の場ごとのモデル化形式と、意味の場間の対応関係についてもモデル化するための形式も包含する

2. 意味の場の状態

1. 意味の場の状態は、モノとモノ間の意味的関係で記述される
2. モノは、それを特徴づける複数の”値が確定した特徴値（Instantaneous Property Value）”を伴い、複数の特徴値が紐づけられたモノのことを”インスタンス（Instance）”と呼ぶ
3. インスタンスに紐づいた特徴値の内の一つ、あるいは二つ以上の特徴値の値によって、その意味の場における、モノの”一意性（Identity）”が記述される
4. モノ間の意味的関係は、二つのインスタンスの間で記述され、両端に文字列によるフ”レーズ（Phrase）”を伴う
5. 両端に意味的フレーズを伴った二つのインスタンスをつなぐ意味的関係のことを、”意味的リンク（Semantic Link）”と呼ぶ
6. 意味的リンクは、接続されたインスタンスのどちらか一方に紐づけられた特徴値の値が、その反対側のインスタンスの一意性を記述する特徴値（複数も可）の値と同じとすることによって、その存在が表現される
7. 意味の場の状態を記述したモデルを、あえて示す場合は、”インスタンスモデル（Instance Model）と呼ぶ
    1. インスタンスモデルは、概念情報モデルで定義された概念クラスをひな型とするインスタンスと、リレーションシップをひな型とする二つのインスタンスの間の意味的リンクの集まりで記述される

    2. インスタンスモデルを構成するインスタンスの特徴値の値は確定している
8. ”意味の場の状態”は、文脈的に誤解が生じない場合は、単に”意味の場”と呼ぶこともある
9. 意味の場において、モノが意味を伴って立ち現れていることについて、概念モデリングでは、それぞれの意味の場に対する、”意味の構造（Semantic Schema）”、つまりスキーマモデルがあって、その制約に従って立ち現れているものとする
    1. 人間の思考・認識は、意識的・あるいは無意識のうちに、あらかじめ獲得しているスキーマを物差しとして意味付けをしている
    2. どのスキーマを紐づけるのか、その直前で止めること＝現象学のエポケーである
10. 意味の構造を記述するスキーマのことを、”概念ドメイン（Conceptual Domain”、または、単に”ドメイン”と呼ぶ
11. 概念ドメインを厳密に記述するモデルを”概念モデル（Conceptual Model）”と呼ぶ

3．概念モデル

1. 概念モデルは以下の3種類のモデルからなる

    1. 概念情報モデル（Conceptual Information Model）
    2. 状態モデル（State Model）
    3. データフローアクションモデル（Data Flow Action Model）

2. 概念情報モデルは、インスタンスモデルの概念・分類を記述したモデルであり、”概念クラス（Conceptual Class）”と”リレーションシップ（Relationship）”で記述する

    1. 一定の複数の”特徴値（Property）”の組をもち、かつ、後で述べる”リレーションシップ（Relationship）”が紐づけられた概念・分類のことを””概念クラス（Conceptual Class）”と呼ぶ
        1. インスタンスモデルのインスタンスは、ただ一つの概念クラスをひな型として存在する
        2. インスタンス間の意味的リンクは、ただ一つのリレーションシップをひな型として存在する
    2. 概念クラスは、そのドメインで唯一の名前を持つ
    3. 特徴値は、概念クラス内で唯一の名前を持ち、ドメインで定義された”データ型（Data Type）”で、その値が制約される
        1. 一つの特徴値、あるいは複数の特徴値の組の値によって、概念クラスをひな型とするインスタンスの一意性が記述される
            1. その組は複数あって構わない
    4. リレーションシップは、二つの概念クラスの間に定義される
        1. 概念モデリングの”リレーションシップ”は、数学的に厳密に言えば、”リレーション（Relation）”である
            1. ”リレーションシップ”と言う用語を使っているのは、この単語が単に一般的なソフトウェア工学、例えば、リレーショナルデータベースなどでの慣用句だからという理由にすぎない
        2. それぞれのリレーションシップを区別するために、便宜上、ドメインでただ一つのインデックスを付与する
        3. 両端に、多重度、意味的フレーズを必ず記述する
            1. 多重度は、0以上の自然数である
                1. 概念モデリングでは、これまでの経験と意味構造定義の必要十分性から、”1”、”0..1”、”1..＊"、”＊”の四種類で十分であるとする。
            2. 意味的フレーズは任意のテキストとする

        1. 二つの概念クラスの間（ただ一つの概念クラスの場合も同様）に二つ以上のリレーションシップが定義されている場合、両端の意味的フレーズの組の値は異なっていなければならない
        2. 一つのリレーションシップに付与される両端のフレーズは必ず異なったテキストでなければならない
4. インスタンスモデルの意味的リンクは、ただ一つのリレーションシップをひな型とする
    1. 結果として、インスタンスモデルにおける意味的リンクの両端の意味的フレーズは、リレーションシップの意味的フレーズに一致する
    2. さらに、あるインスタンスに張られた意味的リンクの数も、リレーションシップの多重度に従う
    3. 概念クラスの幾つかの特徴値によって、インスタンスの意味的リンクの存在を表現する特徴値がどれであるかが決定される
        1. その特徴値のことを参照特徴値と呼ぶ
        2. 参照特徴値は、リレーションシップで紐づけられた反対側の概念クラスの識別子の特徴値の組のどれかに一致する
        3. 参照特徴値のデータ型は対応する識別子の特徴値のデータ型に一致する
5. リレーションシップは、二つの概念クラスの間、つまり、"二項関係（Binary Relationship）が基本ではあるが、意味構造の分類から、概念モデリングでは、以下の二つのリレーションシップを追加する
    1. "Is-a"
        1. ある概念クラスSのインスタンスが一つ存在した場合
        2. 概念クラスC1、C2、C3、...（ただし有限とする）の内、駆らなずどれかの概念クラスのインスタンスがただ一つだけ存在する
        3. 必要であれば、S をスーパー側（Super Side）、C*n*をサブ側（Sub Side）と呼んで区別する
        4. ※ オブジェクト指向プログラミングの継承や拡張に似ているが全く異なる意味なので注意
    2. "Associative"
        1. ある概念クラス、O、Tの間にリレーションシップが定義されていて、そのリレーションシップをひな型として立ち現れている意味的リンク一つづつに対して、概念クラスAのインスタンスがただ一つ、必ず紐づいている
        2. この場合では、Aを"関連クラス（Associative Class）"と呼び、参照特徴値は、Aに紐づく
6. 二つの概念クラスO、Tがあるものとし、それぞれの側の多重度と意味的フレーズを、O側に付与されたもの：*mo*,*Po*、T側に付与されたもの：*mt*、*Pt*とする
    1. 意味の場の状態は、概念情報モデルのリレーションシップの定義から導出される命題文と、それらの命題文の連結で記述できる。それぞれのリレーションシップの種別ごとの基本命題文は以下の通りである
        1. "Binary"の場合
            1. *o*:O *Pt* *t*:T
            2. *t*:T *Po* *o*:O
        2. "is-a" の場合
            1. O をスーパー側、Tをサブ側とすると、以下の通り
            2. *t*:T is-a *o*:O
        3. "Associative"の場合
            1. 関連クラスをAとすると、以下の通り

        2. *a*:A *Po* *o*:O
        3. *a*:A *Pt* *t*:T
    2. 論理学によれば、命題文は無意味か、もしくは有意味な場合True、または、Falseであるが、意味の基盤が意味の場であるから、有意味な命題文は、ある一つの意味の場においてのみ構成でき、かつ、概念情報モデルのリレーションシップの定義から導出された命題文が、実際のインスタンスモデルに一致しているなら True、一致していなければ、Falseである
    3. また、概念情報モデルから導出できない命題文もまた無意味である　1. これを数学的に表現すると、ゲーデルの不完全性定理と同型であろう
    4. リレーションシップをひな型とし、二つのインスタンスの間に張られる意味的リンクの内、あるインスタンスの恒等射と同等な意味的リンクは存在しないものとする
        1. 一つのリレーションシップのみに限らず、複数のリレーションシップの足し合わせによる、大域的に生じる場合も存在しないものとする
        2. “存在しないものとする”とは、概念モデルをスキーマとするインスタンスモデルにおいて、そのような状態は許されないことを意味する
7. データ型は、数学の形式に従った、数値、テキスト（記号の一次元の羅列）、離散値を基本型とし、モデル化対象の意味の場にふさわしい名前と値域を持つ
    1. 概念モデリングの基本データ型（primitive data type）は以下のとおりとし、特徴値、イベント引数、データフローに紐づく変数のデータ型として使うことができるものとする
        1. 数値型
            1. 整数（Integer）
            2. 実数（Real）
            3. 複素数（Complex）
            4. 数値型には、二項演算子（Binary Operator：＋、－、×、÷、％、＞、≧、<、≦、==、!=）が伴う
        2. 文字列型
            1. 一つ以上の有限の長さの任意の記号の羅列（String）
            2. 文字列型には、結合（concat：+）、等価（equality：== | !=）、部分文字列抽出（substring）、検索（contains、indexOf）、長さ（length）、置換（replace）という演算子が伴う※ 文字列を、なるべく自由モノイドとして扱いたい
        3. 一意性が保証される何らかの値（Unique-ID）
        4. 参照先と同じデータ型であることを示す（Reference）
        5. 他に、データフローモデルの中でのみ利用可能なデータ型を以下とする
            1. インスタンス参照（Instance Reference）
            2. インスタンス参照セット（Instance Reference Set）
            3. イベントインスタンス参照（Event Instance Reference）
    2. 意味の場において、その概念モデル作成の際には、基本型である数値型や文字列型をベースに、その意味の場における特有の意味、あるいは制限を付与した、データ型を定義できる、そのデータ型のことを”ユーザー定義データ型（User Defined Data Type ※ 本来的には、FoS Specific Data Typeだが）”と呼ぶ
    3. ユーザー定義データ型は、意味の場において一意の名前を与え、必要であれば、その値域を定義する
        1. ユーザー定義データ型は、以下の3種類に分類される

        1. 基本データ型の拡張
           数値型、文字列型をベーすとする
        2. 列挙型（Enumerated Data Type）
           値が、2つ以上の異なる有限個の文字列のどれかである
        3. 構造型（Structured Data Type）
           子要素として、そのデータ型において一意の名前が付与されてかつ、基本データ型（数値型、文字列型に限る）、または、ユーザー定義データ型のデータ型を持つ変数を持つデータ型
    4. 一部のデータ型の詳細な意味は、別の意味の場で定義されている場合もある
        1. 例えば時刻や日時、もっと言えば数学的に厳密な意味をもつ実数などがその良い例である
        2. その仕組みは判らなくても、その形式のみ、日常で有用である
        3. 概念モデリングにおいては、その仕組みや成り立ちなどは曖昧、不明でもよしとするが、利用するために必要な最低限の形式だけ（基本型と値域）を定義すれば十分であるとして扱う
        4. 必要であれば、別の意味の場の概念モデルで記述されていることを明示すること

3. 状態モデル

    1. 状態モデルは、概念クラスに対して、ただ一つだけ記述する
    2. 状態モデルは、以下の道具立てからなる
        1. イベント（Event）
        2. 状態（State）
        3. 遷移（Transition）
    3. 状態モデルもまた、スキーマであり、インスタンスモデルを構成する各インスタンスは、状態モデルをひな型とした"状態機械（State Machine）"を伴っているとする
    4. イベントは、意味の場において生起する出来事の分類に対応する
        1. イベントは、一つの状態モデルにおいて唯一のインデックスと、意味的な名前を持ち、かつ、一つ以上の引数を伴う
        2. 引数はドメインで定義されたデータ型によって規定される
        3. 厳密に言えば、イベントは、意味の場におけるモノ（Object）であり、引数を特徴値と見立てれば、インスタンス（Instance）でもあるが、すでに存在するインスタンスによって消費されるものとする
            1. 消費とは、後程述べるように、遷移を引き起こすことに相当する
            2. 消費されたイベントは消滅する
            3. 紐づけるインスタンスが存在しない場合は、概念クラスのインスタンスが一つ生成されることをもって消費とする
        4. イベントは、分類・概念であり、実際に意味の場に生起したイベントを明示したい場合は、"イベントインスタンス（Event Instance）"と呼ぶことにする
    5. 意味の場で立ち現れているインスタンスで、そのひな型である概念クラスに状態モデルが定義されている場合は、その状態モデルで定義された、どれか一つの状態にあるとする
        1. UML の状態モデルは、階層化など非常に複雑なモデルが記述可能であるが、意味の構造を記述する概念モデリングにおいては不必要であり、階層化などは行わない
    6. 状態は、その意味の構造、概念クラスの状態として相応しい名前を付与し、その名前は状態モデルにおいて、一意でなければならない

7. インスタンス（の状態機械）がある状態にある時、その状態からの遷移に紐づいたイベントインスタンスが生起した場合、遷移先への状態遷移が生じる
8. 後で詳細を述べるデータフローアクションモデルが、それぞれの状態に対して紐づき、そのモデル全ての実行が完了した時点で、遷移が確定する
9. 遷移確定後は、インスタンスモデル全体の状態が概念情報モデルの制約に従っていなければならない
    1. ただし、遷移から状態確定までの間は一時的に制約が敗れることは容認される

4. データフローアクションモデル

1. データフローアクションモデルは、複数の入力データから出力データに変換する基本プロセス（バブルともいう）の組合せで記述される
2. 基本プロセスは、インスタンス・意味的リンクの生成・消滅、特徴値の参照更新、算術計算、条件判定などで、使用可能な基本プロセスは、概念モデリングにおいて、特定の意味の場に依存せずに決まっている
    1. 基本プロセス部品は以下の通りである
        1. インスタンスの生成
            1. 概念情報モデルで定義された概念クラスのインスタンス（データ型は Instance-Set）を出力する
                1. 意味の場としてインスタンスの存在と共に確定するべき特徴値がある場合は、それらの設定に必要な入力データを伴なうこととする
            2. 識別子特徴値で、かつ、データ型が Unique-Id の特徴値は、自動的に値が確定するものとする
        2. インスタンスの削除
            1. 入力データとして指定されたインスタンスを削除する、出力データはない
        3. 意味的リンクを張る
            1. 概念情報モデルに定義されたリレーションシップに基いて、インスタンス間に意味的リンクを張る
            2. binary, is-a 1.入力データとして指定された二つのインスタンスの間に意味的リンクを張る
            3. binary-associative
                1. 入力データとして、二つのインスタンスと、意味的リンクに関連付くインスタンスの、計3つを使って、意味的リンクを張る
        4. 意味的リンクを切る
            1. 入力されたインスタンスに張られた意味的リンクのうち、指定したリレーションシップの意味的リンクを切る
        5. 特徴値の参照
            1. インスタンスと特徴値の名前を入力とし、該当する特徴値の値を出力データとする
        6. 特徴値の更新
            1. インスタンスと特徴値の名前、更新する値のデータフローを入力とし、該当するインスタンスの特徴値を更新する
        7. 概念クラスをひな型とするインスタンス、または、インスタンスセットの取得
            1. 後述の論理演算、及び、その組み合わせによる条件付けを伴なえる
            2. この基本プロセスの出力データのデータ型は、インスタンス参照（Instance-Ref）、または、インスタンスセット（Instance-Set）である

8. あるインスタンスを起点にリレーションシップを辿った、特定の概念クラスのインスタンス、また、インスタンスセットの取得
    1. 後述の論理演算、及び、その組み合わせによる条件付けを伴なえる
    2. この基本プロセスの出力データのデータ型は、インスタンス参照（Instance-Ref）、または、インスタンスセット（Instance-Set）である
9. 算術演算
    1. 二つの算術演算可能な入力データから一つの出力データを生成する二項演算子
        1. 出力データのデータ型は、それぞれの二項算術演算子によって異なる
        2. 入力データのデータ型がスカラー型の場合、出力データのデータ型は、より濃度の濃いデータ型とする
    2. Instance-Set については、そのデータ型を持つ入力データから、その要素数（Integer）を出力データに変換する一項演算子
10. 論理演算
    1. 二つの論理演算可能な入力データを、一つのBoolean型の出力データに変換する二項論理演算子
    2. Boolean型の一つの入力データを、一つのBoolean型の出力データに変換する一項演算子
11. イベントインスタンスの生起
    1. 状態モデルで定義されたイベントを基に、その引数群に必要な入力データセットと、以下の詳述する受取先を入力として、イベントインスタンスを出力する。データ型は、EventInstance-Refとする
12. External Entity Function
    1. 一つ、あるいは二つ以上のデータを入力とし、出力無し、あるいは一つの出力データに変換する
    2. 詳細は後述
13. 論理演算を基にしたデータフローの部分空間分割
    1. 一つの Boolean型のデータを入力とし、アクションモデルの部分空間のみを実行対象とする
    2. プログラミング言語に例えるなら、IF 〜 [ELSE IF 〜] ELSE 〜 に相当する
14. Instance-Set の要素の同時並行実行
    1. アクションモデルの部分空間に対して、Instance-Set の各要素を入力データとして供給する

3. それぞれのプロセスは、入力として定義されたデータが使用可能になった時点で起動し、記述された出力データに変換され、その出力を入力とする別のプロセスがそのデータを利用可能になる
4. プロセスの実行には一定の有限時間がかかるものとする
5. 日付や時刻、ISBN Codeなど、その本来の仕組みは別の意味の場で記述される物事に付随する操作を、基本プロセス部品に加え、定義し、アクションデータフローモデルで利用できる
    1. 操作のまとまりのことを"外部実体（External Entity）"と呼ぶ
    2. 外部実体は、概念ドメインで唯一の名前を持つ
        1. 名前は、モデル化対象の意味の場から見て、ふさわしいと思われる概念ドメインの名前をつければよい
    3. 外部実体に紐づいて定義されるプロセス（操作）のことを"External Entity Function"と呼ぶ
        1. External Entity Function は、外部実体に対してただ一つの名前をもつ

        1. データ型で定義された複数の引数（無くても可）を入力として持つ
        2. データ型で定義された値を出力を持つ（無くても可）
    4. 概念モデリングにおいて、なんだかよくわからないけど、使い方だけは明確になっているような操作のモデル化は、これを使えば良い
6. 状態に紐づいたデータフローアクションモデルを構成する基本プロセスを組み合わせて、概念情報モデルをスキーマとしたアクションモデルの構築が可能であり、そのモデルのことを"ドメインファンクション（Domain Function）"と呼ぶ
7. ドメインファンクションは、インスタンスモデルの状態のクエリー（人間の認識過程を厳密に記述したものと同等）で利用可能である
8. エンジニアリングで概念モデリングを活用する際、ある意味の場のシミュレーションを行う時のインスタンスモデルの初期状態を構成するときにも使用される
9. データフローアクションモデルは、ブリッジ（Bridge）、変換（Transformation）の記述でも利用される

5. 外部実体（External Entity）

    1. モデル化対象の意味の場において、概念情報モデルの語彙と例レーションシップを持たないが、概念モデルを記述するのに必要な概念を外部実体（External Entity）とする。
    2. 外部実体は、意味の場において一意の名前で区別される
    3. 外部実体は、一つあるいは複数のデータ型が紐づく場合がある
    4. 外部実体は、前述の通り、複数の External Entity Function を伴なう
    5. 外部実体に付随するデータ型の論理演算子・算術演算子は、それぞれ External Entity Function に対応する

6. 一つの意味の場に紐づく概念モデルで記述されたスキーマはただ一つである、しかし、そのスキーマを元に立ち現れる意味の場は無数である

    1. 概念・分類を元にした意味構造である概念モデル（スキーマ）と、それをひな型に立ち現れる無数の意味の場は、厳密に区別すること

7. ユニバース（Grothendieck Universe）

    1. モデル化対象の意味の場の意味構造と変化のルールは、概念情報モデル、状態モデル、データフローベースのアクションモデルによって、スキーマ圏として記述される ドメインファンクションは、そのスキーマ圏に対する性質である
    2. スキーマ圏とドメインファンクションを併せた記述が、モデル化対象の意味の場のユニバースである

概念モデリングは、概念モデルによって記述されたドメイン、スキーマによって、意味の場の状態、およびその変化が起こることを、"意味を伴って現象する"こと、としています。

4. 記述されたモデルの妥当性

これまで述べてきた概念モデルの定義から、意味の場の状態を記述するインスタンスモデルと、スキーマである概念ドメインモデルは、数学的に、圏としての記述が可能です。よって、インスタスモデルの圏$\mathcal{I}$と概念ドメインの圏$\mathcal{C}$は圏同値の関係です。モデル化対象の意味の場のそもそもの状態が圏として記述でき、かつ、それを圏$\mathcal{S}$とします。圏論を応用することにより、検証が可能であることと、検証の手順が導き出されます。

1. 概念モデルとして定義した概念クラスを、意味の場の状態空間の閉包上の関数ベクトル空間の基底とし、リレーションシップによる概念クラス間の接続を多項式とし、ラプラシアンの条件を満たすとすれば、いかなる意味の場に対しても、意味の場の状態を記述する概念モデルがただ一つだけ存在することが数学的に成り立つ
2. さらに、概念モデルは、モデル化対象の意味の場がとりうる任意の状態（閉包の内部の開集合の状態）を記述できる
3. 二つの圏の間に圏同値な対応がある場合、二つの圏は同一だと結論できる
4. よって、圏$\mathcal{I}$とモデル化対象の圏$\mathcal{S}$が自然同値であることを示せば、その妥当性の検証が可能である
5. ある時点における圏$\mathcal{I}$のモデルから圏$\mathcal{C}$の制約（イベントの生起）に従って変化させた状態を圏$\mathcal{I}'$とする
6. モデル化対象の意味の場において、対応する変化を適用する前、した後の圏をそれぞれ、圏$\mathcal{S}$、圏$\mathcal{S}'$とする
7. 任意のイベントの生起に対し、圏$\mathcal{I}$と圏$\mathcal{S}$、圏$\mathcal{I}'$と$\mathcal{S}'$が自然同値であるなら、概念モデル圏$\mathcal{C}$は妥当なモデルである

この検証過程は、基本的には、物理学をはじめとする自然科学において、ある仮説（法則・理論）を実証的に検証する時の過程と同じ形式であり、更に言えば、物理学を、物質界を特徴づける変数群と、変数間の関係性を数式によって表現する体系とすれば、その形式は、概念モデリングの体系と自然同値であると言えるでしょう。

また、“3. 概念モデル”で記述した、インスタンスのアイデンティティ、リレーションシップの制約、状態モデルの“Can't Happen”などの、概念モデルというスキーマを記述するための文法（Syntax）に従って概念モデルを記述したとしても、いインスタンスモデルが、それらの制約を満たさない不正な状態に陥るような概念モデルはあり得ます。この場合、概念モデリングでは、一つの意味の場については、以下のどれかであると考えます。

1. 対象とした意味の場を正確にモデル化できていない
2. 対象とした意味の場が、そのような不具合を抱えている、つまり何らかの破綻に陥るような可能性が潜在している

どちらも現実の意味の場の実際の状況と比較することにより判断が可能であり、後者の場合は、ビジネス的な意味の場であれば、そのビジネスモデル・プロセスは潜在的に問題を抱えていることを意味し、制御の意味の場であれば、その制御もまた何らかの問題を抱えていることを意味します。つまりは、概念モデリングを適用することにより、現実世界の意味の場の不具合を発見したことになります。

ただし、後述の“9. ソフトウェアシステムの実装”に従って変換・生成されたソフトウェアにおいて、その実行中における概念モデリングの制約を満たさないインスタンスモデルの状態の発生は、上の2種類が解決されている場合において、変換先の実行プラットフォームに何らかの不具合（例えば、メモリエラーや誤動作など）が生じていることを意味します。

5. 概念モデルの概念モデル

概念モデリングもまた、一つの意味の場です。よって、概念モデリングの概念モデルも記述が可能です。

1. 概念モデリングという意味の場を対象とした概念モデルを圏$\mathcal{M}$とする
2. 全ての意味の場の概念モデルは、圏$\mathcal{M}$のインスタンスモデルである
3. よって、全ての圏$\mathcal{C}$と圏$\mathcal{M}$は圏同値である
4. ドメインファンクションにより、圏$\mathcal{M}$をスキーマとして、それぞれの意味の場の概念モデルの記述要素を全て系統的に取り出せる

5. 概念モデリングの概念モデルは、概念モデルをひな型とする意味の場の状態を記述するための道具立て（インスタンスモデル）の概念を含む
6. 加えて、状態モデルをひな型とする状態機械の実行セマンティクス、データフローアクションモデルをひな型とするアクションの実行に関するセマンティクスの概念も含む

概念モデルは、フッサールの現象学、カントの認識の形式、フレーゲ・ウィトゲンシュタインの言語哲学をベースに、その形式を意味付けて、圏論、関数空間論、ホモトピー論によって厳密化・体系化を行っています。

概念モデリングの形式は、人間の認識・思考の基本形式（の一部）と同型であると考えています。

6. 意味の場間の対応付け

人文系のビジネスモデルのなかには、社員や顧客といった概念クラスが考えられます。その意味の場において現象しているインスタンスが存在する場合、それが現実の世界で起きていることと自然同値であるなら、そのインスタンスに対応する、生物である人間が必ず一人対応します。更には、生物である人間の身体は、物質界の素粒子の塊が対応しています。

この様に、現実世界で並立している複数の意味の場は、それぞれ意味的には無関係であり、独立してはいますが、それぞれの意味の場上での立ち現れ同士が対応関係を持っているものです。

概念モデリングは、現実世界を記述するモデリング体系なので、この対応関係を形式的に記述する体系を持っています。

1. 二つの意味の場を、それぞれ、$\mathcal{S}$、$\mathcal{T}$とする
2. 二つの意味の場の間の対応関係の記述を"ブリッジ（Bridge）"と呼ぶ
    1. 概念モデルと同様、ブリッジはスキーマであり、それぞれの意味の場のスキーマ、つまり、概念ドメインを厳密に記述した概念モデルを、その対応関係を記述するための対象とする
    2. よって、意味の場で現象しているインスタンス同士の対応付けは、ブリッジというスキーマを制約として、その対応付けが記述される
3. 対応関係は、現実の世界の現象の形式からの考察から、以下の３種類に分類する
    1. 強制的
        1. 物質界と人文系意味の場の対応関係や数学的原理などによる強制的な対応付け
        2. 全ての意味の場の状態変化には、概念モデリングの概念モデルをスキーマとする意味の場がそれぞれ同時に立ち現れていると考えられる
           この対応付けは強制的である
    2. 恣意的
        1. ビジネスモデルや思想など、人間の思考における対応付け
        2. 自然科学における法則・理論も、観測結果を人間が発見したスキーマに当てはめているともいえる
            1. この考え方からすると、その対応付けは恣意的であるともいえる
            2. しかし、数学的には、スキーマはただ一つであることから、自然の原理と同型であるといえる
    3. 論理的
        1. ブリッジによる対応付けと、それぞれの意味の場のスキーマの制約を掛け合わせた場合に生じる対応付け
            1. 例えば、物質界から立ち上っている人間社会の意味の場において、社会活動が活性化し、化石燃料を多く消費した場合、物質界と人間社会の有意味な状態変化によっ

て、他の強制的な対応付けが生じて、温暖化が進むなどの物質界の変化が生じるような、対応付けのこと

4. ブリッジは、以下の二つの道具立てで記述する
    1. 拡張された概念情報モデル
    2. データフローアクションモデルによる対応関係の記述
5. ブリッジにおける拡張された概念情報モデルは以下のとおりとする
    1. 概念情報モデルは、意味の場毎の意味構造を記述するためのモデルであり、そもそも無意味な二つの意味の場の対応付けを、そのままでは記述できないため、形式のみ、活用する
    2. ある意味の場のインスタンスと別の意味の場のインスタンスに対応関係がある場合、それぞれのひな型となっている概念クラスを使って、その対応関係を記述する
    3. 概念クラスは、別の意味の場における概念であることを明確にするため、"射影クラス（Projection Class）"と呼ぶ
        1. 一つの意味の場を対象とした概念情報モデルにおける概念クラスは、その定義によって意味の構造を記述するものであるが、射影クラスは、単なるインスタンスのひな型であるとしてブリッジでは扱われる
    4. 異なる意味の場由来の射影クラスのインスタンス間に対応関係がある場合、"ファンクター（Functor）"で表現する。リレーションシップと同様に、両端に多重度（4種類）を記述するが、意味的構造ではなく、単なる対応関係であるため、意味的フレーズは不必要である
    5. 対応関係を定義するために必要であれば、ブリッジ（という意味の場）における概念クラスを定義する場合もある。
6. データフローアクションモデルによる対応関係の記述は以下のとおりとする
    1. 拡張された概念情報モデルに記述された、射影クラス、概念クラス、ファンクターをスキーマとし、対応付けに必要な双方のモデル要素のクエリーアクション
    2. 対応付ける双方の状態を記述するアクション
        1. 対応付けは、対応付けの種別によって以下の2種類がある
            1. 強制的な対応付けの場合は、双方同時にアクションが実行されるものとし、この時のアクションの実行には時間がかからないものとする※ 演算子は、"<=>"を使用する※ ファンクターが紐づけられるときは、"<= '*Name of Functor*' =>" のようにその紐づけを明記する
        2. 恣意的な対応付けの場合は、引き起こす側のアクションを左に、それに応じて変化する側のアクションを右に記述し、"=>"で結合する
    3. これらの記述形式により、以下のような対応付けを表現可能とする
        1. ある特徴値が更新される => イベントインスタンスが生起する
        2. ある意味的リンクが張られる => ある特徴値が更新される
        3. ある状態遷移が生じる => インスタンスが生成される
        4. データフローアクションモデルの基本プロセスは、複数の意味の場に対応できるよう拡張される
7. 有意味な命題文の制約
    1. 3-2-5で記述した、一つの意味の場における、命題文の有意味性はあくまでも一つの意味の場に限って成り立つ
    2. 現実世界においては、複数の意味の場が並立していて、かつ、ブリッジによる対応関係が定義されている場合は、以下の二種類も含めなければならない
        1. その対応関係
        2. それぞれの意味の場の概念モデル（スキーマ）

3. 命題文が現実の世界で有意味で、かつ、True であるためには、全ての意味の場で整合性が無ければならない

命題文の有意味性については、

"36. 意味が付与された実在とは、そして、有意味な命題とは？"

で詳しく解説しているので、ご一読ください。

7. 記法について

概念モデリングにおいては、モデルとモデル図を厳密に区別します。あくまでも世界を認識する形式が重要であり、どう表記するかは、その形式の本質に沿っていれば、何でもよいという立場をとります。モデルの記述においては、モデリングの国際標準であるUML表記法で図示してもいいですし、YAML や JSON など、更にはある程度形式化された自然言語でも構いません。一方で、複数の人の間でモデルを共有するためには、何らかの記法に則って記述されたモデル図（図、テキスト両方を含む）が必要なのは間違いありません。

概念モデリングは、モデル、モデル図について、次のような立場をとります。

1. 記法は一つの意味の場のスキーマであり、記法に従って記述されたモデル図は、その意味の場の状態（インスタンスモデル）である
2. 記法の意味の場のスキーマ圏を $\mathcal{L}$ とする
3. ある意味の場に対して構築された概念モデルを図示化するということは、以下のプロセスと同等である
    1. 概念モデリングの概念モデル（以下、メタモデルと略称する）をスキーマとして、記述方法を系統的に取り出す
    2. あらかじめ決められたルールに則って、$\mathcal{L}$ をスキーマとするインスタンスに変換する
    3. ルールは、メタモデルと$\mathcal{L}$の間の対応関係を記述するブリッジと、ブリッジをスキーマとするインスタンスレベルでの対応付け、および、モデルから$\mathcal{L}$のインスタンスに変換する、データフローアクションモデルで記述される
4. 実用性を考慮した場合、記法は、モデリングの本質を余すところなく表現できることが最上ではない
5. 記法ごとに、見やすさや抜け漏れの防止、直観的理解性、形式的な厳密さ、等のメリットがあり、それであるからこそ、それぞれの記法の価値が生まれる
6. 一方で、それぞれの記法は、より強調される点と省略される点が必ず存在してしまう
7. 記法の選択においては、その点をよくよく考慮し、どれかの記法について、信仰的な思いを持つことなく、用途の目的に応じた最善と思われるものを選択すること

概念モデリングのメタモデルに従って概念モデルをテキスト形式で記述する場合のJSON Schema を

cogito/models/ConceptualModeling at main · kae-made/cogito · GitHub

から公開しているので参考にしてください。ちなみに、ある特定の意味の場の概念情報モデルを元に、そのインスタンスを定義するために定義された言語が、DSL（Domain Specific Language）です。

世の中には数多のモデリング体系がありますが、有意味性の観点からすると、ここで解説した複数の意味の場毎のスキーマを元として、かつ、意味の場間の対応関係という形式と同型でないモデリング体系によって作成されたモデルは、ほとんど無意味だ、というのが私の結論です。

8. 現実世界の描写

これまで説明してきたモデリング体系は、現実世界の在り様を記述することについて、形式としては十分だというのが、現時点での見解です。この見解は、現状の様々な自然科学における法則を概念モデリングの形式で書き換えるべきだという主張ではなく、自然科学において現象のあり様を記述する形式は、概念モデリングの形式と同型だ、ということを意味します。

概念モデリングにおける、データ型、概念クラス、特徴値の名前、および、リレーションシップに付与される意味的フレーズが単なる記号の羅列だとします。実際、それは単なる記号の羅列であり、概念モデルのなかに配置されることによって、その記号の羅列に意味が付与されるのですが。その前提を元に考えると、スキーマたる概念モデルとブリッジの記述は、数学の原理に従ってランダムに幾らでも構成が可能です。現代の最先端の物理学から導き出された驚くべき結論の、ホログラム原理やブラックホールの情報消失問題の解決策からの推察になりますが、数学的にあり得るスキーマとその制約に基づいた一つの意味の場の状態が一定規模以上で存在すれば、数学的にあり得るブリッジが自動的に起動して、別の意味の場の状態が立ち上るのではないかという仮説が成り立つでしょう。

それらを前提として、概念モデリングの形式を使って自然科学のフレームワークを形式化した結果を書いておくことにします。

1. ホログラム原理は、量子ビット情報のスキーマとこの物質界のスキーマ（アインシュタインの時空方程式）と、そのスキーマ間のブリッジが数学的原理に従って可能性として存在している
2. 量子ビット情報の意味の場が存在し始めることにより、ブリッジが自動起動して、我々の時空間が立ち現れている
    1. ブリッジの記述、および、我々の時空間のスキーマが、数学的原理にしたがうこと、時間が等間隔に流れることをベースにしている
    2. よって、その立ち上った物質界の存在である我々の認識において、モノゴトが全て数学的原理に従っていることと、同じ慣性系では時間が等間隔に流れていることが保証される
3. 物質界という意味の場において現象しているモノゴトの結果、数学的可能性としてあり得る生命界の意味の場のスキーマとブリッジが発動し、生命活動という意味が付与された意味の場が立ち現れる
    1. この形式では、物質界と生命界はそれぞれ別の意味の場であり、何故生命現象が生じるのか、その理由を問うことは無意味である、なぜなら、単に数学的可能性として存在するブリッジが起動した結果に過ぎないから
4. 我々の思考は、そのような意味の場のスキーマに従った状態と、ブリッジの発動によって、人間の脳生理学の意味の場が立ち現れて、更に別のブリッジの発動によって、人間の意識や思考が立ち現れる...という一連の意味の場の状態→ブリッジの発動→別の意味の場の立ち上りをへた、意味の場であると言える
5. そのようにして発現している、人間の思考の意味の場において、生命界という、発見した意味の場のスキーマという物差しを使って、単なる物質界の現象を生命界のインスタンスモデルを作っていることになる、それが生命現象の探究であるといえる
    1. 例として物質界と生命界をとりあげたが、形式的には、量子論や、ありとあらゆる自然科学のモデルの探究も同じ形式である
    2. 圏論、関数空間論、ホモトピー論をうまく当てはめれば、概念モデリングの体系・形式は、モデル化対象を必ずモデル化できて、その状態変化も含め、妥当なモデルはただ一つだけであることが保証される
    3. 必ず1つだけあることが担保されていれば、人間の思考を駆使して発見したスキーマは、どんな表現体系で記述されていたとしても、自然界のあり様と一対一であり、その妥当性が保証される
    4. 繰り返しになるが、意味の場が異なれば、その間に単なる対応関係はあるものの、ごった煮にした有意味な命題文は構成できない単に、二つの意味の場のスキーマとブリッジがあるのみで

あるそれをある別の意味の場の状態への写像とする試みにおいて生じるのが、”創発”や”因果”である。

6. 古今東西、様々な哲学的言説があるが、それらの哲学的言説もまた、概念モデリングの形式で表現が可能である
    1. むしろ、概念モデリングの形式に従ってそれぞれの言説を記述すれば、それぞれの弱点が明確化されたり、より厳密な分析が可能になると思われる
    2. 構造主義は、意味の場のスキーマを定義し、恣意的な対応付けの基、現実社会の複数の意味の場の物事を、定義したスキーマによる意味付けの基、インスタンスモデルを作っていると言えるこの様なインスタンスモデルの作成は、ストーリーを作るのと同型である
7. この物質界は、無限次元のヒルベルト空間の閉包上の関数ベクトルを規定とする多項式である汎波動関数をスキーマとした量子ビット情報という意味の場（$\mathcal{Q}$）から、グリーン関数（G）によってアインシュタインの時空方程式をスキーマとする時空間（$\mathcal{E}$）という意味の場が立ち上っている、これが概念モデリングの形式をホログラム原理の現象を適用した記述である
    1. $\mathcal{Q} \xrightarrow{G} \mathcal{E}$の形式によって、この時空間が立ち上っているのであれば、一つの（$\mathcal{Q}$）から複数の$\mathcal{E}$が立ち上がっている可能性は否定できない
    2. ある二つの意味の場のスキーマがあり、そのスキーマに従う意味の場の状態が一定条件を満たせば、変換ルールであるグリーン関数がただ一つだけ存在することが数学的に証明されているそうであれば、同じ$\mathcal{Q}$からアインシュタインとは異なる時空方程式をスキーマとする全く別の時空間も立ち現れている可能性も否定できない
8. 繰り返しになるが、ある意味の場の中から別の意味の場を認識することはできない
    1. 認識とは、意味の場を圏とした場合に、関手は定義できないということ
    2. そもそも認識とは意味付けが付随するわけだから、別の意味の場の認識そのものが無意味である
    3. さらに言えば、新実在論において、「複数の意味の場が同時並行的に立ち現れている」と言った場合、それは正しくはない”同時”とは、すでに時空間という意味の場を前提としてしまっている意味の場が異なれば、同じ時空間において、と考えること自体が無意味である二つの意味の場が、時空間が意味を成す意味の場と対応付けがある場合にのみ、同時性や空間の距離などに関する有意味な命題を構成できるそれ以外においては、二つの意味の場の間の距離や時間的な順序について、有意味な比較は一切できない
9. 人文的な意味の場もまた自動的に立ち現われているかもしれない
    1. 自然数や実数は、その形式が数論として定義されると、それぞれの数は無限に存在しうる
    2. 意味の場のスキーマ（概念モデル）は、数学的原理を基にした概念モデリングの形式に従って記述されるのであるから、こちらも無限のスキーマが存在しうる
    3. 加えて、ある二つのスキーマの間で記述される変換ルールを伴なうブリッジ（グリーン関数に相当）もまた、二つのスキーマ共に特定の条件を満たせば、ただ一つだけ存在するため、ブリッジもまた無数に存在しうる
    4. この物質界が Ads/CFT のような数学的原理の形式によって、勝手に立ち現われているなら、他のあらゆる（変換ルールを伴なう）ブリッジもまた、勝手に発動し、ある意味の場から別の意味の場を立ち現わせていることは否定できない
    5. 以上の形式的な立ち現われが成り立つのであれば、物質界の意味の場におけるモノゴトの実在性と同じレベルで、人文系のありとあらゆる意味の場についても、実在すると言って良いだろう
        1. 京極夏彦氏の百鬼夜行シリーズで語られている「社会がシステムを共有していれば呪いが成り立つ≅この世に不思議なことは何もないのだよ」は、意味の場間の対応付けも含めれば、その実在性を形式的に記述できるであろう

    2. 京極夏彦氏の"豆腐小僧"の妖怪たちも、ある意味の場において実在していると考えてよいだろう
10. これまで書いてきた一連の説明を読んで腑に落ちない読者は沢山いるかもしれない
    1. この正解が地続きの一つの巨大な意味の場であるという呪縛に囚われているなら、概念モデリングの体系は一切理解できないであろう
    2. 概念モデリングは以下の形式に従ってこの世界が存在していることを主張するだけの体系であることを今一度咀嚼していただきたい
        1. この世界は複数（無限）の意味の場の並立である
        2. 意味の場はそれぞれスキーマが紐づき、状態はそのスキーマの制約に従う
        3. 意味の場間にはスキーマを使った対応関係が存在する
        4. 対応付けが強制的ならば、ある意味の場の状態があれば、自動的に別の意味の場の状態が構成される
    3. 一見すると、スキーマ（概念モデル）が、プラトンのイデアに相当するように見えるかもしれない
        1. しかし、実在するのは、スキーマの制約のもとに現れている意味の場の状態であり、実在そのものと実在の形式は厳密に区別して扱わなければならない
    4. 数学的原理から、一つの意味の場を記述する妥当な概念モデルはただ一つである、しかし、それを人間が記述できるとは限らない
        1. 正しい答えが存在することと、正しい答えが見つかることは同義ではない
        2. 実際、この世界をモデル化しようとすると、一見シンプルな意味の場でも、実際には複雑で奥深いため、正しい答えの全体像をモデル化することは困難であろう
        3. ただし、物理学における、相対性理論や量子論という厳密な法則よりもニュートン力学が実用上役立つのと同様に、本来の概念ドメイン全てを網羅するモデル全体の一部を切り出したモデルでも、実用上は十分である
            1. 漸近的に本来の式に近づいていく多項式と同型であろう
            2. 概念モデル（概念ドメイン＝スキーマ）だけでなく、ブリッジもまた同様である
            3. ちなみに、人災などでよく使われる"想定外"とは、拾い出していなかった、概念クラスやリレーションシップ、対応付けが威力を発揮してしまった状態のことである

SaG of Modeling for Real World｜Knowledge & Experience｜noteの一連の記事で書いている通り、今のところ、形式的に記述できそうにない言説には出会っていません。逆に、言説ごとの弱点や改良点が明確になり、より明晰になるだろうと考えています。この一連の確かめは、概念モデリングの体系が哲学的観点から見た妥当性の検証の一環として位置付けています。

9. ソフトウェアシステムの実装

冒頭で書いた通り、概念モデリングがベースとするのは、組込み制御システムを開発するためのエンジニアリングの方法論である"Shlaer-Mellor法"です。この技法の実用例は、サンフランシスコの鉄道管理システム、ロスアラモスのシンチレーター制御、F16ミッションコントローラ、OA機器制御ソフトウェアなどがあります。

Refine & Redefinitionを経た概念モデリングは、当然のことながらそれら過去の事例も包含するし、今様のIoT・Digital Twins システム・ソリューションも含めた、ありとあらゆる IT Solution 開発を視野に入れています。加えていえば、継続的にバズワードとして出てくる、DX、いわゆるデジタルトランスフォーメーションの実現においても活用できるのは当たり前田のクラッカー。

1. IoT・Digital Twins、DX、ITソリューションは人文系のビジネスモデルでその要件が定義される
2. ビジネスモデルと呼ばれるものは、大抵の場合、数個の意味の場からなる

3. 概念モデリングは、その意味の場の抽出・発見をしながら、それぞれの意味の場の概念モデルと、意味の場間の対応付けモデルを構築していく
4. 現実の世界においては、それらの意味の場は、物質界などの意味の場との強制的な対応付けが存在し、このケースにおいては、物質界は、人文系の意味の場を成り立たせるための実行環境・エンジン（以後、"実行プラットフォーム"と呼ぶ）という位置づけであると考えられる
5. ソフトウェアの構築は、現実世界における物質界の代替の意味の場を設定し、恣意的に人文系の意味の場の状態を、その実行プラットフォームの意味の場のスキーマ、より具体的に言えば、プログラミング言語がそれに対応するが、そのスキーマに従ったインスタンス群を構成することに相当する
    1. Digital Twins の本質は、現実世界の人文系の意味の場に紐づいたスキーマを概念モデルとして記述し、そのスキーマの基に実在している現実世界の人文系の意味の場の状態と、デジタル空間上に保持した意味の場の状態が自然同値になっていること、である
    2. 世に言うDX（デジタルトランスフォーメーション）も現実世界の人文系の意味の場と自然同値な意味の場をデジタル化することが基本であるから、ここで述べている原則が満たされていなければ、そのシステムは無意味である
6. よって、意味の場間のブリッジを記述すれば、それをルールとして変換することにより、想定したソフトウェアの実行環境に合致したプログラムコードが生成される
    1. 現状、単に生成系AIを使って生成したコードの妥当性の検証方法が懸案となっているが、概念モデリングの体系を生成系AIに加味すれば、検証可能なコードが生成可能になるはず
7. 現実世界においては物質界との対応付けから論理的に導かれる実行上の制約、例えば、同時に処理可能なトランザクションや応答反応などの制限が確実に存在するのに対し、ソフトウェアの実装は、物質界という実行プラットフォームの意味の場を、コンピューティングテクノロジーの意味の場に置き換えることであり、実行に関わる制限は、使用するコンピューティングテクノロジーに依存する
    1. 制御系も含め人文系の意味の場が意図通り動作するためには、それらの制約を満たす、コンピューティングテクノロジーを選択しなければならない
    2. 実行プラットフォームに対する制約、およびその集まりのことを"非機能要件"と呼ぶ
    3. 非機能要件は、実装したい意味の場の概念モデルの拡張として記述する
        1. 例として、いくつか挙げておく
            1. 生成できるインスタンスの総数
            2. 同時に実行可能な状態機械の数
            3. イベントの生起から状態遷移が確定するまでの時間
8. ソフトウェアで実装したい意味の場（$\mathcal{B}$）から、選択したコンピューティングテクノロジーの意味の場（$\mathcal{P}$）の間で定義した変換ルール（R）を実行して、ソフトウェアコードを生成する、このプロセスを、"変換による実装"、あるいは、Shlaer-Mellor 法からの伝統に基づいて"リカーシブ・デザイン（Recursive Design）"と呼ぶ
    1. 変換ルール（R）は、数学的には、グリーン関数と同値である
    2. 同様に変換ルール（R）は、妥当なものがあれば、それはただ一つである
    3. $\mathcal{B} \xrightarrow{R} \mathcal{P}$は、形式的には、最先端の物理学で論じられているホログラム原理と同型であるといえる
9. 変換ルールは、ブリッジの記述用に拡張されたデータフローアクションモデルによって記述される
    1. 実用上の観点から、データフローアクションモデルのセマンティクスを守りつつ、テキスト変換用に用意されたスクリプト言語を使用する
        1. 代表的なテキスト変換用スクリプト言語は、BridgePoint の Archtype や、Visual Studio の T4 Template である
        2. ブリッジによる変換ルールのモデルと、変換用スクリプト言語によって構築されたモデルコンパイラは双対性の関係にある

10. これら、ブリッジの定義をベース・拡張した、ある意味の場と、その非機能要件から選択された別の意味の場、その間の変換ルールを使った、意味の場間の変換のことを、"変換（Transformation）"と呼ぶことにする

10. 意識について

意識に対し、概念モデリングの形式の適用を試みます。意識については、当たり前のことですが、奥深く、まだ不明なことも多いので、試案レベルであることをご留意ください。

1. 意識もまた、概念モデルをスキーマとした意味の場である
    1. "生命現象界（$\mathcal{LP}$）"、"脳神経ネットワーク（$\mathcal{NN}$）"などの意味の場の状態から、"意識（$\mathcal{SC}$）"という意味の場が、そのスキーマの制約の元に立ち現われている
        1. 一つの人体ごとに意識の意味の場が立ち現われているため、他人の意識（の意味の場の状態）を直接認識することはできない
        2. 一つの人体に、同時に複数立ち上っている可能性も否定できない
            1. それがフロイトのいう無意識界、かもしれないし、多重人格なのかもしれない
    2. $\mathcal{SC}$ は、感覚系（$\mathcal{SS}$）との間に定義されたブリッジの基に、感覚器官の状態変化が、メタモデルをスキーマとする意味の場の状態変化やイベントインスタンスの生起を引き起こす
        1. その状態変化やイベントインスタンスの生起によって引き起こされる状態モデルに従った変化が意識の流れである
        2. その過程で、$\mathcal{SC}$ と"筋肉を動かす系（$\mathcal{MS}$"の間のブリッジにより、物質界への影響がしょうじる
    3. モノゴトの認知過程は、ドメインファンクションの実行である
        1. 意識内の思考の流れを省察するに、概念モデリングのメタモデルの状態を、より適切な意味の場の状態として整理することに相当するだろう
            1. この過程は、ある程度複数のスレッドの並行操作とそれらの連なりだと思われる
            2. ここで言及している"意味の場"とは、あくまでもメタモデルのスキーマで定義された"概念ドメイン"という概念クラスのインスタンスを制約とする"意味の場"という概念クラスのインスタンスであることに注意
                1. 人間の意識・思考は、あくまでも一つの意味の場の状態であり、概念ドメイン、意味の場を認識するとは、単にその状態を構成するインスタンスを並べているという事である
                2. ただし、その概念ドメイン、意味の場の状態は、実在する意味の場のスキーマと状態と自然同値になりうるということである
    4. 思考においてなされる想像や思考実験は、全てのこの形式に従って記述できるだろう
        1. 思考内の意味の場と実在する意味の場が自然同値になりうるときで、かつ、実在する意味の場が物質界であり、意味の場ごとで命題文が有意味でかつ、対応付けの制約を満たす場合にのみ、この世界でも実在が可能である
            1. 簡単に言えば、人間が体一つで空を飛ぶ空想はできるが、実際には飛べなかったり、ワンピースのゴムゴムの実の能力者は、実際には存在しないということ
                1. ただし、京極夏彦氏の小説に出てくるように、社会のシステムとして成り立つ（＝対応付け制約も満たしている）場合には、呪術廻戦のような世界が成り立つかもしれない
            2. 思考実験による物理法則の妥当性の検証は、この形式で言いかえられるであろう

このセクションは、まだ全く検討が足りない状況で書いています。意識・思考の概念モデルも、どのようなモデルになるか、今のところ不明です。数学的原理からすれば、モデルはただ一つだけあるはずなのです

が、ただ一つだけ必ず妥当なモデルが存在することと、実際にそのモデルがどんなものかは別のお話です。

現時点での私の関心の中心は、個々の意味の場のモデルとはどんなものか、ではなく、あくまでも概念モデリングの形式の妥当性の方なので、実際のモデリングは、どなたかにお任せします。（無理かw）

11. コミュニケーション

"7. 記法について" で記述した通り、モデルそのものと、テキストや図によって表現されたものは、同一ではありません。36. 意味が付与された実在とは、そして、有意味な命題とは？｜Knowledge & Experience 図8を参考にしてほしい。しかし、誰かの思考を別の誰かと共有するためには、何らかの表記に基づいた記述が必要になります。誰かの思考＝モデルと、それを表現したもの＝モデル図が本質的に同一ではないのに、人間はどうやって相互理解を行うのでしょう。この点について概念モデリングから導き出される形式を書いていきます。

1. 概念モデリングの体系から、人間の意識は、同じ意味の場のスキーマをひな型として、それぞれの人間ごとに立ち上っている意味の場の状態であると、形式化される。
2. ある意味の場から別の意味の場を認識することは不可能であるため、人は自分の意識のみ認識が可能であり、他人の意識を認識することはできない
    1. もしかすると、一人の人間の中に複数の意識が立ち上っている可能性も否定できない
    2. 複数立ち上っていても、意識は別の意味の場に立ち上っている意識を認識することはできない
    3. この場合、同じ人体という意味の場との対応関係はあるはずであるから、もしも別の意識が立ち上っているとしたら、その意味の場の状態変化に論理的な対応付けが発動した結果の人体の意味の場の状態の変化を通じて、別の意味の場を発見することはあるかもしれない
3. 他者はモデル図を見ることにより、図8とは逆の方向で、他者自身が有する（すでに発見済みの）スキーマを元に意味の場の状態を構築する
4. 他者による完全な理解は、用いられたスキーマが、モデル図の作成者が用いたスキーマと一致していることである
    1. モデル図を一度見ただけで、一致が見られることは保証されない
    2. 一致のためにはコミュニケーションが必要である
        1. コミュニケーションは双方向だけでなく、一方向の場合もある
5. 万人に共有されたスキーマ、それが”普遍性”である
    1. ”普遍性”をもつスキーマに従った意味の場の状態のみ”客観性”をもつ
    2. スキーマの一致していない思考は全て、単なる主観である
    3. 数学や物理学、法体系、思想など、あらかじめ普遍性のあるスキーマが共有されている場合、双方向のコミュニケーションがなくとも、理解が成り立つ場合もあるであろう
6. コミュニケーションには複数の意味の場が伴い、複数あることによって、スキーマの一致が促進される
    1. モデルが図として表現された場合
        1. 図で使われる四角形や線、アイコンなどのインスタンスモデルとそのスキーマ、および、表現したい意味の場のスキーマとのブリッジ
        2. 図形の配置などの意味の場
    2. モデルがテキストとして表現された場合、文法・語彙のインスタンスモデルとそのスキーマ、および、ブリッジを基本とし、以下も意味の場として付随すると思われる
        1. 書体、タイポグラフィーの意味の場
        2. ラング、ソシュール、言語ゲーム
    3. 対面の会話の場合
        1. いわゆる、ボディランゲージなど

    2. 会話の場
    3. レトリック、音韻
4. いずれの場合にも、コミュニケーションを行う場、も関係し、それらが増えれば増えるほど、お互いの理解は早まる
5. ”人”対”人”のコミュニケーションと、”人”対”生成系AI”のプロンプティングは、形式的には同値であるだろう

コミュニケーションについては、まだまだ検討の余地がある段階ですが、大枠は妥当であろうと考えています。言葉によるコミュニケーションは奥が深く、多くの意味の場とそれらの間の対応付けがで記述されるのでしょう。この方面は私の専門の範囲を超えているので、その筋の専門家にゆだねたいと考えています。